Analysis of Algorithms: Lecture 1

Important Short Questions (25) · Answers from the lecture slides

Stars	Meaning
★★★	Most important, prepare first
★★	May come
★	Sometimes comes, read once

PART 1: MOST IMPORTANT (Q1–Q10)

Q1. What is an algorithm? ★★

An algorithm is a well-defined step-by-step procedure that takes some **input** and produces some **output**. It is a sequence of computational steps that transform the input into the output, and a tool for solving a well-specified problem.

Q2. What are the ways to specify an algorithm? ★★

1. **Natural language:** simple steps in plain English. Example (addition): Step1: read a. Step2: read b. Step3: c = a + b. Step4: display c.
2. **Pseudocode:** a mix of natural language and programming language. Example: c = a + b, then write c.
3. **Flowchart:** the algorithm shown as a diagram using shapes and arrows.

Q3. Write the features (properties) of an algorithm. ★★★

1. **Finiteness:** it should end after a finite number of steps.
2. **Definiteness:** each instruction should be clear.
3. **Input:** valid input should be clearly specified.
4. **Output:** it should give one or more valid outputs.
5. **Effectiveness:** steps should be simple and basic.

Tip: remember F-D-I-O-E

Q4. Differentiate between Priori and Posteriori analysis. ★★★

Priori Analysis	Posteriori Analysis
Theoretical estimation of resources required	Done after implementing the algorithm on a target machine
Done **before** implementation	Done **after** implementation
Does not depend on the machine	Depends on the machine

Q5. Differentiate time and space complexity. ★★★

Complexity is a function that describes the efficiency of an algorithm in terms of **time and space**.

Time Complexity	Space Complexity
Amount of **time** an algorithm needs to run	Amount of **memory** an algorithm needs to run
Expressed as an order of magnitude	Expressed as an order of magnitude

Space complexity = constant size + variable size

Q6. Define Big-O notation. ★★★

Big-O shows the **upper bound** of an algorithm's running time, i.e. the **maximum** time it can take.

Definition: f(n) = O(g(n)) if there exist a constant c > 0 and n_0 such that **f(n) ≤ c·g(n)** for all n > n_0.
Example: $10n^3 + 4n^2 - 4n + 5$ = **$O(n^3)$** (ignore constants and smaller terms)

Q7. Define Omega (Ω) and Theta (Θ) notations. ★★★

- **Omega (Ω):** shows the **lower bound**, i.e. the **minimum** time an algorithm takes. f(n) = Ω(g(n)) if **f(n) ≥ c·g(n)** for all n > n_0.
- **Theta (Θ):** shows **both upper and lower bound**. f(n) = Θ(g(n)) if **f(n) = O(g(n)) and f(n) = Ω(g(n))**.

Tip: O = ceiling (≤), Ω = floor (≥), Θ = both

Q8. Explain worst, best and average case. ★★★

- **Worst case (Big-O):** the **maximum** time. The algorithm will never take longer than this, whatever the input.
- **Best case (Omega):** the **minimum** time. The input for which the algorithm runs fastest.
- **Average case (Theta):** the **expected** time when the input is random.

We usually focus on the **worst case** because it is easier to analyze and gives a guarantee.

Q9. Explain the types of algorithms. ★★

1. **Probabilistic:** uses random choices whose probability is known and controlled. Example: Randomized Quick Sort
2. **Heuristic:** based on optimism (a good guess), with little theory behind it. Error cannot be controlled, only estimated.
3. **Approximate:** gives an answer as accurate as required. We decide how much error is acceptable (e.g. 2 or 8 decimal places).

Q10. Write the functions in order of increasing growth rate. ★★

$c < \log N < \log^2 N < N < N \log N < N^2 < N^3 < 2^N$

Function	Name
c	Constant
log N	Logarithmic
$\log^2$ N	Log-squared
N	Linear
N log N	N log N
N^2	Quadratic
N^3	Cubic
2^N	Exponential

Tip: top to bottom = fastest to slowest; 2^N is the slowest

PART 2: MAY COME (Q11–Q19)

Q11. Differentiate between an algorithm and a program. ★★

Algorithm	Program
Step-by-step method to solve a problem	Algorithm written in a programming language
Written in English, pseudocode or a flowchart	Written in C++, Java, etc.

A program is a set of instructions which the computer follows to solve a problem.

Q12. Why do we need algorithm analysis? ★★

- A working program is not enough; it may be **inefficient**.
- On a **large data set**, running time becomes a big issue.
- When there are many algorithms for one problem, analysis helps us **choose the best one**.

Q13. What are experimental studies? Write their limitations. ★★

Experimental study: write the program, run it on inputs of different sizes, measure the time (e.g. System.currentTimeMillis()), and plot the results.

Limitations:

1. The algorithm must be implemented first, which may be difficult.
2. Results may not be correct for other inputs that were not tested.
3. To compare two algorithms, the same hardware and software must be used.

Q14. What is theoretical analysis? ★★

- Uses a **high-level description** (pseudocode) instead of real code.
- Running time is written as a **function of input size n**.
- Considers **all possible inputs**.
- Does **not depend** on hardware or software.

Q15. What is the RAM model? ★★

1. has one processor
2. executes one instruction at a time
3. each instruction takes **unit time**
4. has fixed-size operands
5. has fixed-size storage (RAM and disk)

Q16. Why do we use asymptotic notations? ★★

- To represent the **time complexity** of an algorithm.
- To measure running time and **growth rate** for **large input size**.
- To design efficient algorithms that finish in an acceptable amount of time.

Q17. How do we compare algorithms? ★★

1. **Compare execution times?** Not good, because time depends on the computer.
2. **Count statements?** Not good, because the count depends on the language and the programmer's style.
3. **Best way:** write running time as a **function of input size n**, i.e. f(n), and compare these functions. This does not depend on machine or programming style.

Q18. Explain rate of growth with an example. ★★

For large n, **small terms don't matter**, only the biggest term matters.

Example: buying an elephant and a goldfish: Cost = cost of elephant + cost of goldfish ≈ **cost of elephant**.

Similarly: $\mathbf{n^4 + 100n^2 + 10n + 50 \approx n^4}$, so both have the **same rate of growth**.

Q19. Define little-o and little-omega. ★

- **Little-o:** $f(n) = o(g(n))$ if $f(n) = O(g(n))$ but $f(n) \neq \Theta(g(n))$. It bounds only from the **top**.
- **Little-omega:** $f(n) = \omega(g(n))$ if $f(n) = \Omega(g(n))$ but $f(n) \neq \Theta(g(n))$. It bounds only from the **bottom**.

PART 3: SOMETIMES (Q20–Q25)

Q20. What is Algorithmics? ★

The science of studying and evaluating algorithms to **select the best one** for a task, and to **design new algorithms**.

Q21. What do we analyze in an algorithm? ★

1. **Correctness:** does it give the right output for the input?
2. **Amount of work done:** how many basic operations (time).
3. **Amount of space used:** how much memory.

Q22. What is performance analysis? ★

It is the criteria for judging an algorithm. When a problem has more than one algorithm, we find the run time of each and **choose the one that takes less time**.

Q23. What is input size? Give examples. ★

Input size is the number of elements in the input. Examples: size of an array, degree of a polynomial, number of elements in a matrix, number of bits in the input, vertices and edges in a graph.

Q24. Count the primitive operations of arrayMax. ★★

```
Algorithm arrayMax(A, n)                # operations
  currentMax <- A[0]                      2
  for (i = 1; i < n; i++)                 2n
     if A[i] > currentMax then            2(n - 1)
        currentMax <- A[i]                2(n - 1)
  return currentMax                       1
                                  Total = 6n - 1
```

So the running time is **linear, O(n)**. Changing hardware changes T(n) by a **constant factor** only. The **growth rate stays the same**.

Q25. Name the classical multiplication algorithms and the design approaches. ★

- **Classical multiplication:** English, American, A la russe, Divide and Conquer (981 × 1234 = 1210554)
- **Design approaches:** Divide and Conquer, Greedy Method, Dynamic Programming, Search and Traversal, Graph Theory, Linear Programming, Approximation Algorithm, NP Problem

HOW TO STUDY

1. **Part 1 (Q1–Q10):** learn these first and properly (about 45 minutes).
2. **Part 2 (Q11–Q19):** then these (about 30 minutes).
3. **Part 3 (Q20–Q25):** read once at the end (about 15 minutes).

In the paper: if a question says "differentiate", make a **table**, and add **one example** to every answer.